《嵌入式系统》实验报告 5

学号：21013025 姓名：单楚雄 班级：计 211 日期：2024.6.8

成绩：_____ 指导教师：罗飞

实验名称：嵌入式驱动程序的构造及应用程序调试	实验地点：
实验仪器：实验云平台	

一、实验目的：

1. 了解 Linux 驱动程序的结构
2. 掌握 Linux 驱动程序常用结构体和操作函数的使用方法
3. 初步掌握 Linux 驱动程序的编写方法及过程
4. 掌握 gcc 和 gdb 工具进行编译和调试程序的使用方法

二、实验内容：

（一）驱动程序的构造

举例说明一个内核驱动模块程序的构造过程；特别地，加载该模块时，输出“Hello driver-module!”，卸载该模块时，输出“Goodbye driver-module!”。

编写驱动实验代码 dri_arch.c

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE(“Dual BSD/GPL”);
MODULE_AUTHOR(“ECUST EMBED”);
MODULE_DESCRIPTION(“HELLO MODULE”);

static int __init dri_arch_init_module(void){
printk(“This is a simple driver-module!\r\n”);
return 0;
}
static void __exit dri_arch_cleanup_module(void) {
printk(“Goodbye driver-module!\r\n”);
}
module_init(dri_arch_init_module);
module_exit(dri_arch_cleanup_module);
```

2. 编译 make，使用 makefile

2.1 创建 makefile 文件：

```
vim makefile
makefile 内容如下：
Obj-m := hello.o
KDIR:=/home/ecust/samba_share/embed/linux-3.2-FS210/
PWD := $(shell pwd)
default:
  make -C $(KDIR) M=$(PWD) modules ARCH=arm CROSS_COMPILE=arm-cortex_a8-linux-gnueabi-
clean:
  rm –rf *.o *.ko *.order *.mod.c *.symvers
```

所得到的驱动模块文件路径及其文件名为：dri_arch.ko

驱动模块的基本结构包括以下内容：
模块源代码文件：这是用 C 语言编写的源代码文件，其中包含了模块的初始化函数、退出函数以及必要的其他功能代码。源代码文件名为 hello_driver_module.c。
Makefile：用于编译内核模块的 Makefile 文件。Makefile 包含了编译规则和指令，告诉系统如何编译源代码文件以生成内核模块文件（.ko 文件）。在我们的例子中，Makefile 名为 Makefile。
内核模块文件：这是通过编译源代码文件生成的文件，通常具有.ko 扩展名。该文件包含了编译后的二进制代码和模块的元数据信息。在我们的例子中，内核模块文件名为 hello_driver_module.ko。

（二）应用程序的调试

1．构建工作目录并进入工作目录

2. 创建目标实验代码 greeting.c

实验代码内容如下：

```
#include  <stdio.h>
#include  <stdlib.h>
#include  <string.h>

static char buff [256];
static char* string;

int main (void) {
    char welcome[] = “Linux C Tools: GCC and GDB”;
    printf(“This is the experiment to learn %s \n!”, welcome);
    my_print();
}

void my_print()
{
    printf (“Please input a string: ”);
    gets (string);
    printf ("\nYour string is: %s\n", string);
}
```

3. 创建 makefile

makefile 内容如下：

```
Obj-m := hello.o
KDIR := /home/ecust/samba_share/embed/linux-3.2-FS210/
PWD := $(shell pwd)
default:
        make -c $(KDIR) M=$(PWD) modules ARCH=arm CROSS_COMPILE=arm-cortex_a8-linux-gnueabi-
clean:
        rm -rf *.o *.ko *.order *.mod.c *.symvers
```

4. 基于上述 makefile 编译并运行该程序

程序运行错误，主要错误原因是：

```
"makefile" 9L, 239C written
ecust@Ubuntu:~/dritest$ make
make -C /home/ecust/samba_share/embed/linux-3.2-FS210/  M=/home/ecust/dritest modules ARCH=arm CROSS_COMPILE=arm-cortex_a8-linux-gnueabi-
make: *** /home/ecust/samba_share/embed/linux-3.2-FS210/: No such file or directory.  Stop.
make: *** [default] Error 2
ecust@Ubuntu:~/dritest$
```

5. 使用 gdb 进行调试，结合 where, list, print, break, run, set 等调试命令进行调试，记录其过程。

创建 hello.c 文件，编译 hello.c

```
"hello.c" 7L, 124C written
ecust@Ubuntu:~$ gcc hello.c -o hello
ecust@Ubuntu:~$ ./hello
hello,world
hello,world!
hello,world!!
ecust@Ubuntu:~$
```

使用 gdb 进行调试：

gdb hello 启动调试

run 启动程序

```
ecust@Ubuntu:~$ gdb hello
GNU gdb (Ubuntu 7.7.1-0ubuntu5~14.04.2) 7.7.1
Copyright (C) 2014 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.
html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copyin
g"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from hello...(no debugging symbols found)...done.
(gdb) run
Starting program: /home/ecust/hello
hello,world
hello,world!
hello,world!!
[Inferior 1 (process 29628) exited with code 01]
```

break main 在 main 函数开始处设置断点

run 重新运行程序

```
(gdb) break main
Breakpoint 1 at 0x400531
(gdb) run
Starting program: /home/ecust/hello

Breakpoint 1, 0x0000000000400531 in main ()
(gdb)
```

where 显示当前的调用堆栈

```
(gdb) where
#0  0x0000000000400531 in main ()
(gdb)
```

q 退出 gdb

```
(gdb) q
ecust@Ubuntu:~$
```

三、思考：

1．驱动的加载使用主要有哪些方法？它们的差别是什么？

静态加载：

方法：驱动程序在操作系统启动时自动加载。这种方法通常通过在操作系统内核中编译驱动程序代码来实现。

优点：加载时间较快，驱动程序在系统启动后立即可用，适用于关键设备如硬盘驱动器。

缺点：驱动程序会占用系统资源，即使该设备不被使用，系统也会消耗相应的内存和资源。

动态加载：

方法：在需要时手动加载驱动程序。例如，当插入一个新的 USB 设备时，操作系统可以动态加载对应的驱动程序。

优点：只在需要时加载，节省系统资源。支持即插即用设备，方便用户。

缺点：首次加载驱动程序可能会有一些延迟，可能需要额外的配置和管理。

内核模块：

方法：使用命令（如 Linux 系统中的 insmod 或 modprobe）手动加载内核模块，这些模块包含了驱动程序代码。

优点：灵活性高，可以按需加载和卸载。对于开发人员和系统管理员来说，便于调试和更新。

缺点：需要手动管理，可能导致一些复杂的依赖关系和配置问题。

用户空间驱动：

方法：驱动程序运行在用户空间，而不是内核空间，通过特定的接口（如 libusb）与内核通信。

优点：提高系统稳定性，用户空间的崩溃不会影响整个系统。便于开发和调试。

缺点：性能可能较低，因为需要在用户空间和内核空间之间频繁切换。适用于非关键性设备和应用。

即插即用：

方法：操作系统自动检测新硬件并加载适当的驱动程序，无需用户干预。

优点：用户体验好，减少手动配置和安装的复杂性。适用于大多数现代设备。

缺点：有时可能无法找到合适的驱动程序，特别是对于一些较旧或不常见的设备。

不同的驱动程序加载方法适用于不同的场景：

静态加载适合于系统启动时需要立即可用的关键设备。

动态加载和即插即用则更适合于日常使用和外部设备的连接。

内核模块提供了灵活性和调试能力，对于开发和高级管理很有帮助。

用户空间驱动则用于提高系统稳定性和简化开发过程，但可能会影响性能。